

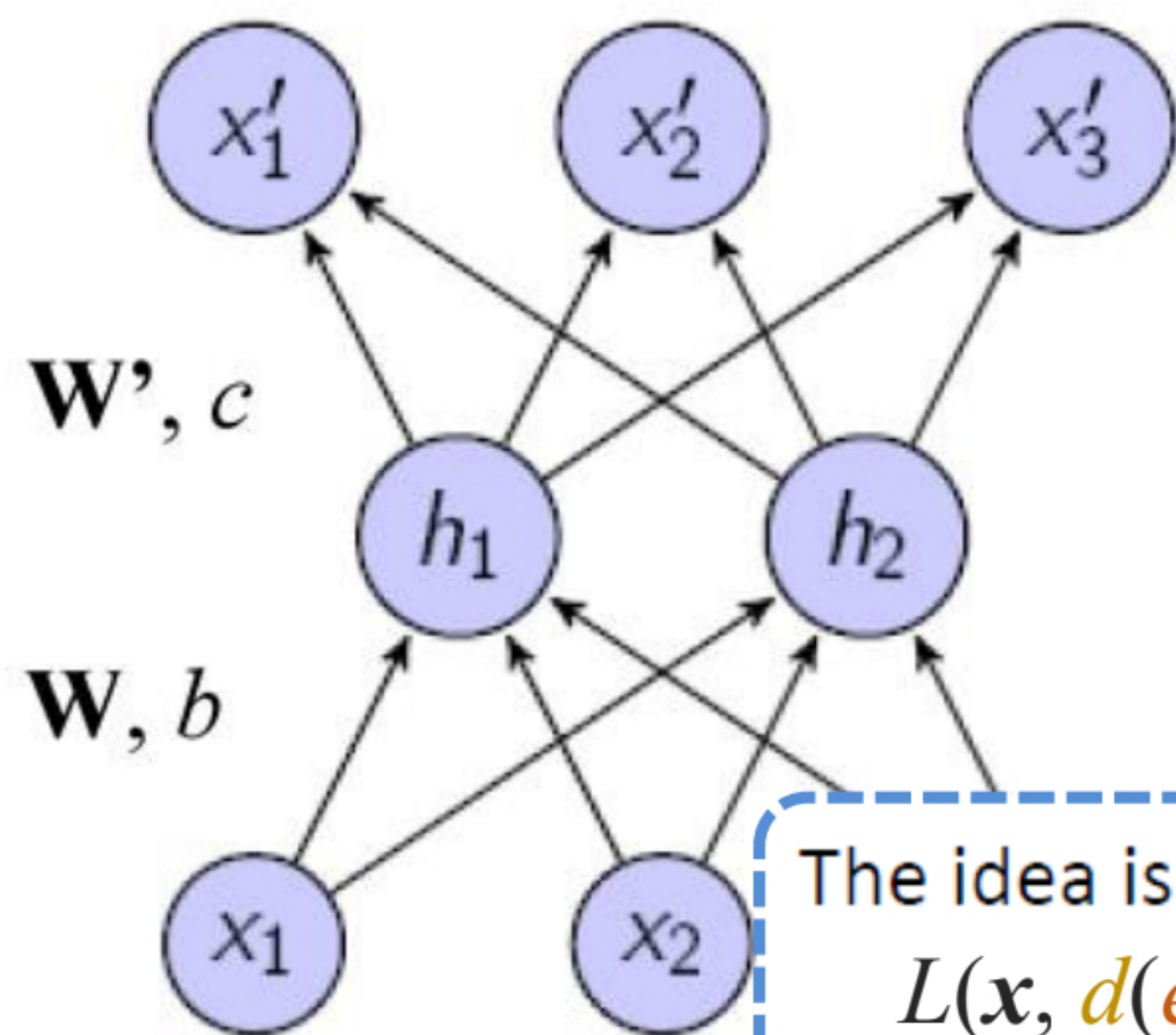
# Representation Learning with Deep Generative Models

**Georgios M. Moschovis (AUEB/Archimedes)**

PhD Student, Athens University of Economics and Business  
Postgraduate Research Fellow, Archimedes Unit / Athena Research Center  
MSc Machine Learning, KTH Royal Institute of Technology

Mail-to: [geomos@athenarc.gr](mailto:geomos@athenarc.gr)

# Introduction to autoencoders



Decoder:  $\mathbf{x}' = d(\mathbf{h})$ , e.g.  $\sigma(\mathbf{W}'\mathbf{h} + c)$ ,  $\mathbf{h} \in \mathbb{R}^n$

Encoder:  $\mathbf{h} = e(\mathbf{x})$ , e.g.  $\sigma(\mathbf{W}\mathbf{x} + b)$ ,  $\mathbf{h} \in \mathbb{R}^m$ ,  $m < n$

The idea is to minimise the loss function,  $L$ :

$$L(\mathbf{x}, d(e(\mathbf{x}))) = L(\mathbf{x}, d(\mathbf{h})) = L(\mathbf{x}, \mathbf{x}')$$

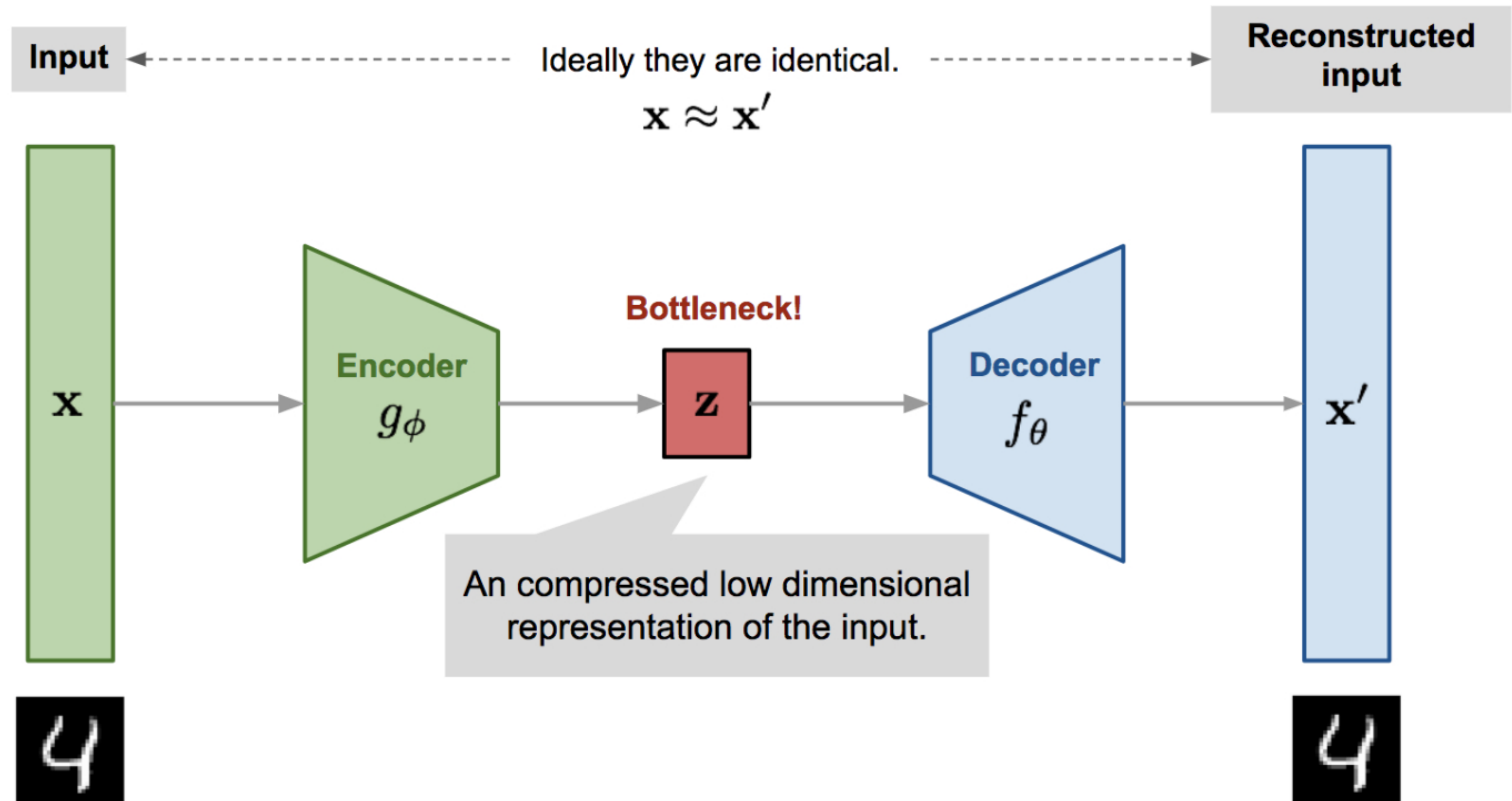
Encourage  $\mathbf{h}$  to produce low reconstruction error (loss  $L$ ) – training with backprop

Reconstruction  $\mathbf{x}' = d(e(\mathbf{x}))$ , e.g.  $\mathbf{x}' = \sigma(\mathbf{W}'\sigma(\mathbf{W}\mathbf{x} + b) + c)$

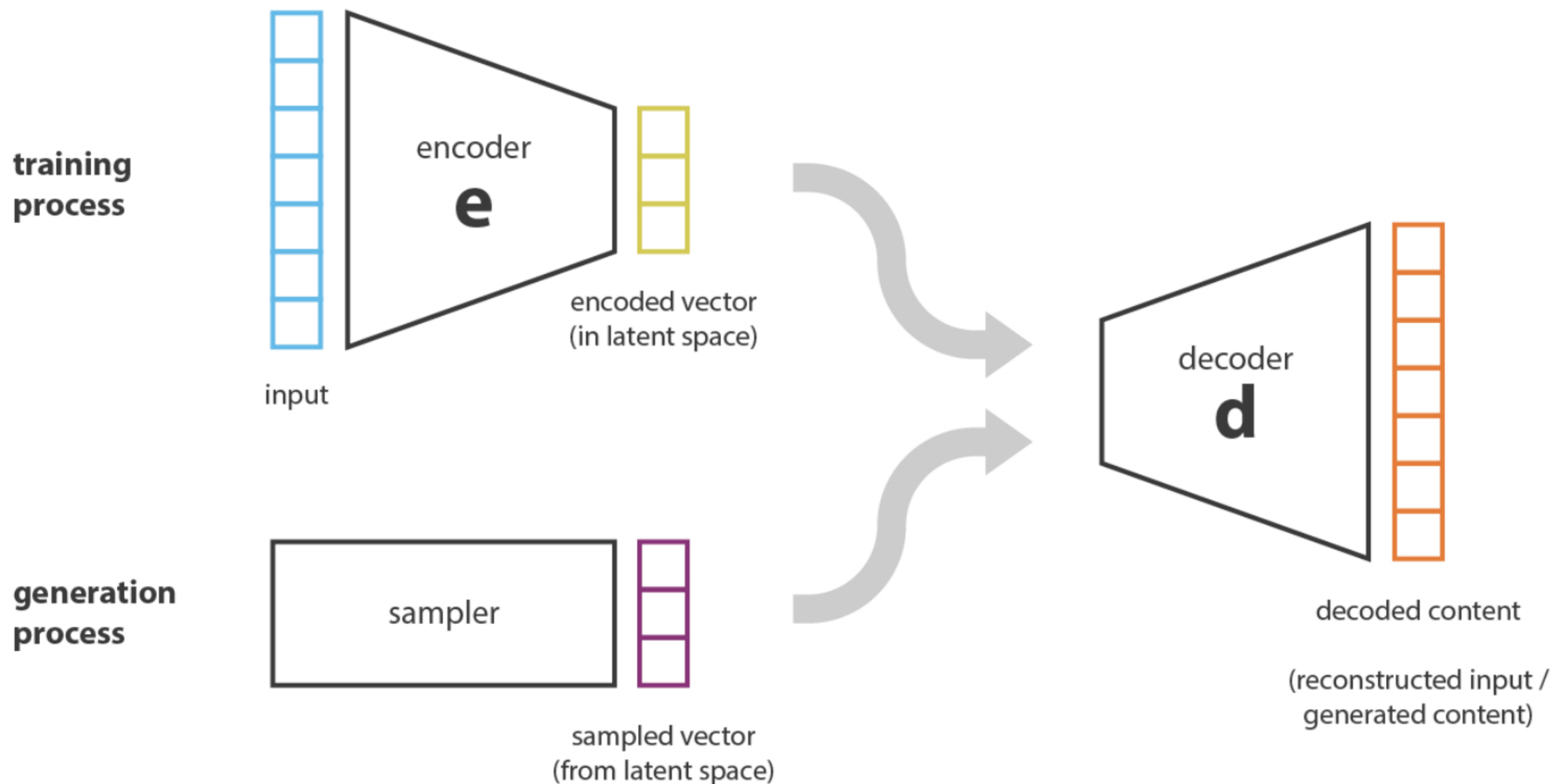
Loss function  $L(\mathbf{x}, d(e(\mathbf{x}))) = L(\mathbf{x}, d(\mathbf{h}))$ , e.g.  $L(\mathbf{x}, d(e(\mathbf{x}))) = \frac{1}{M} \sum_m \|\mathbf{x}^{(m)} - d(e(\mathbf{x}^{(m)}))\|_2$   
(also known as reconstruction loss)



# Introduction to autoencoders



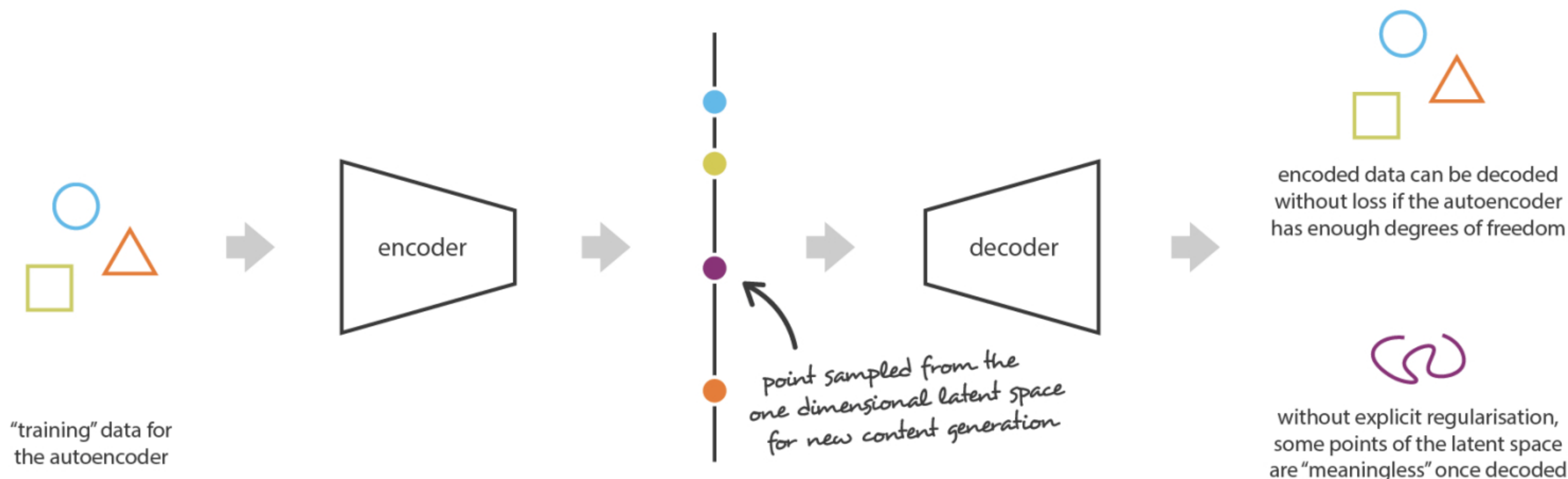
# Generating samples from autoencoders



$$(e^*, d^*) = \operatorname{argmin}_{(e, d) \in E \times D} L(\mathbf{x}, d(e(\mathbf{x})))$$



# Generating samples from autoencoders



$$J(x; \mathbf{W}, \mathbf{W}') = L(x, d(e(x))) + \Omega(h), \quad h = e(x)$$

$$\Omega(h) = \lambda \sum |h_i|$$

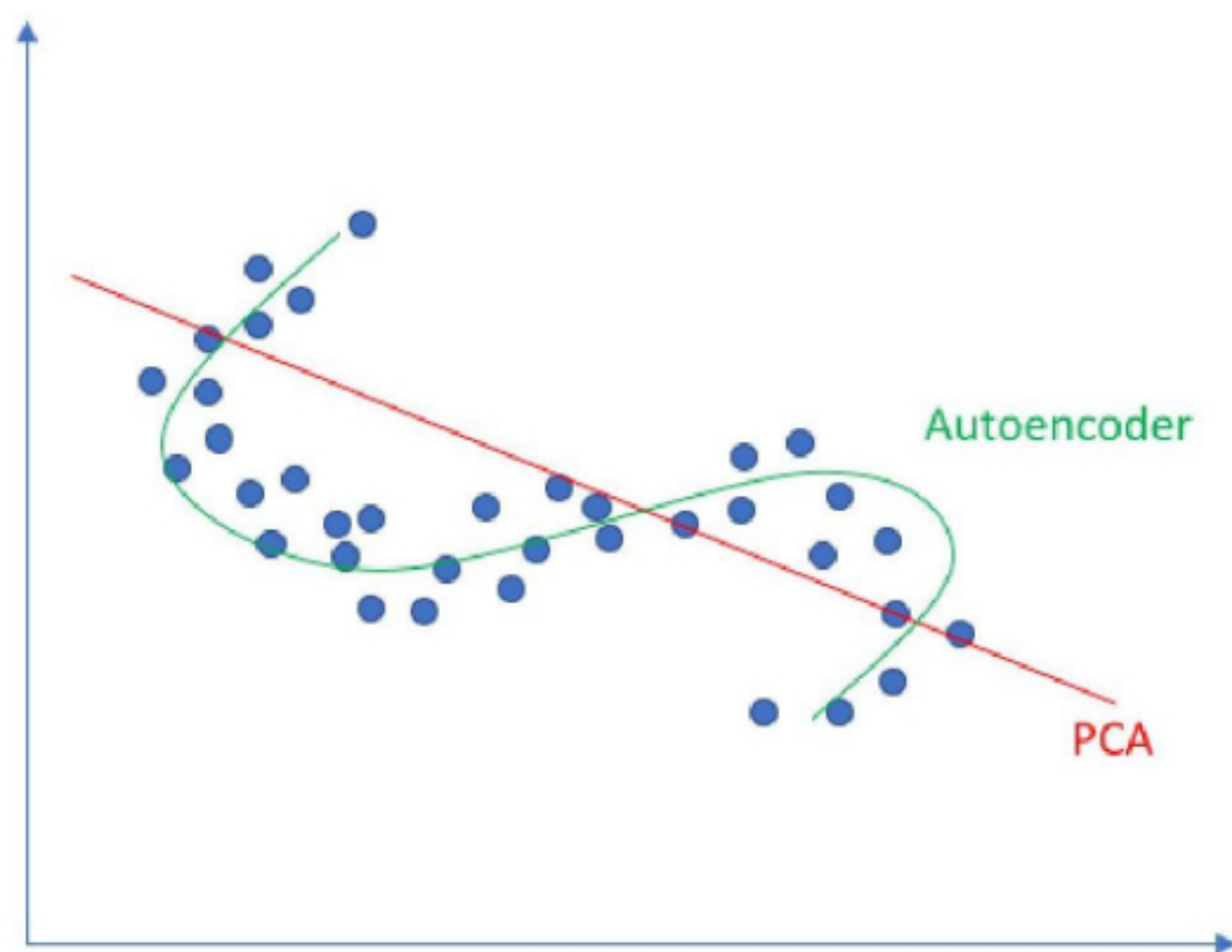
using  $L^1$  norm



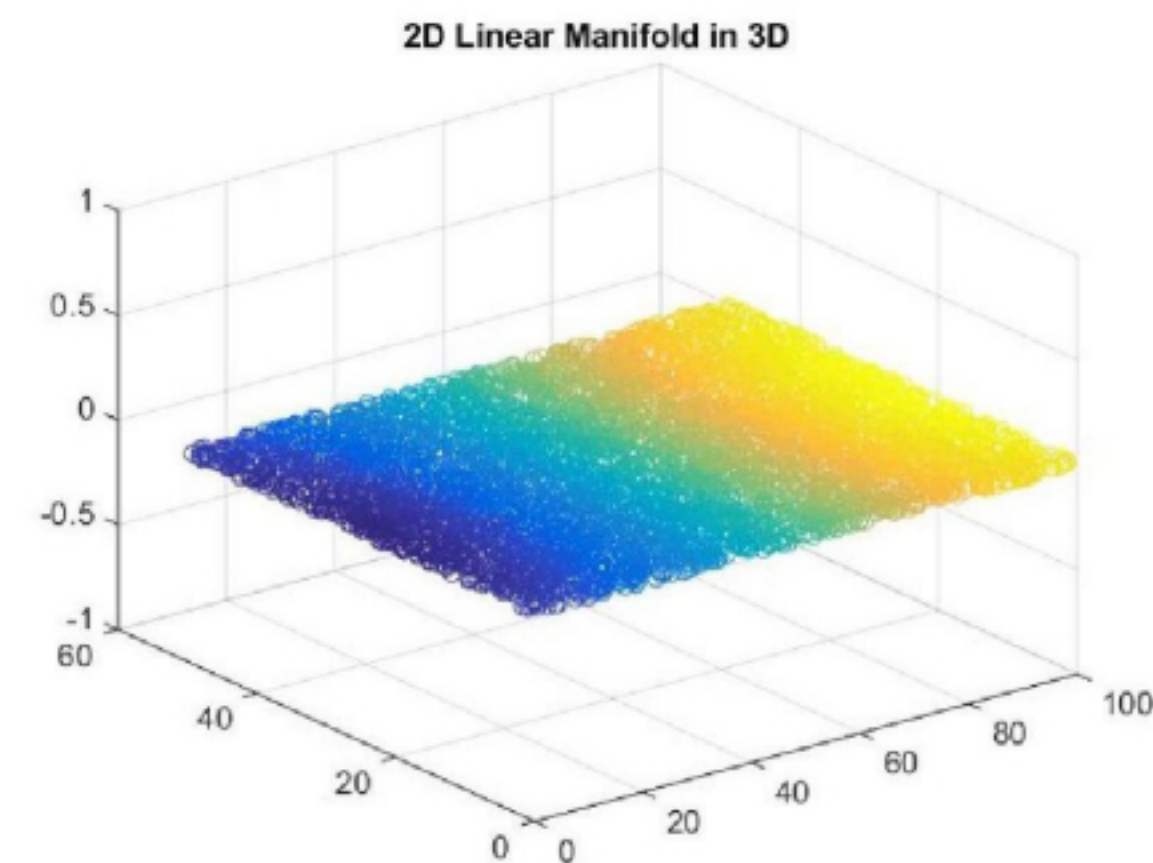
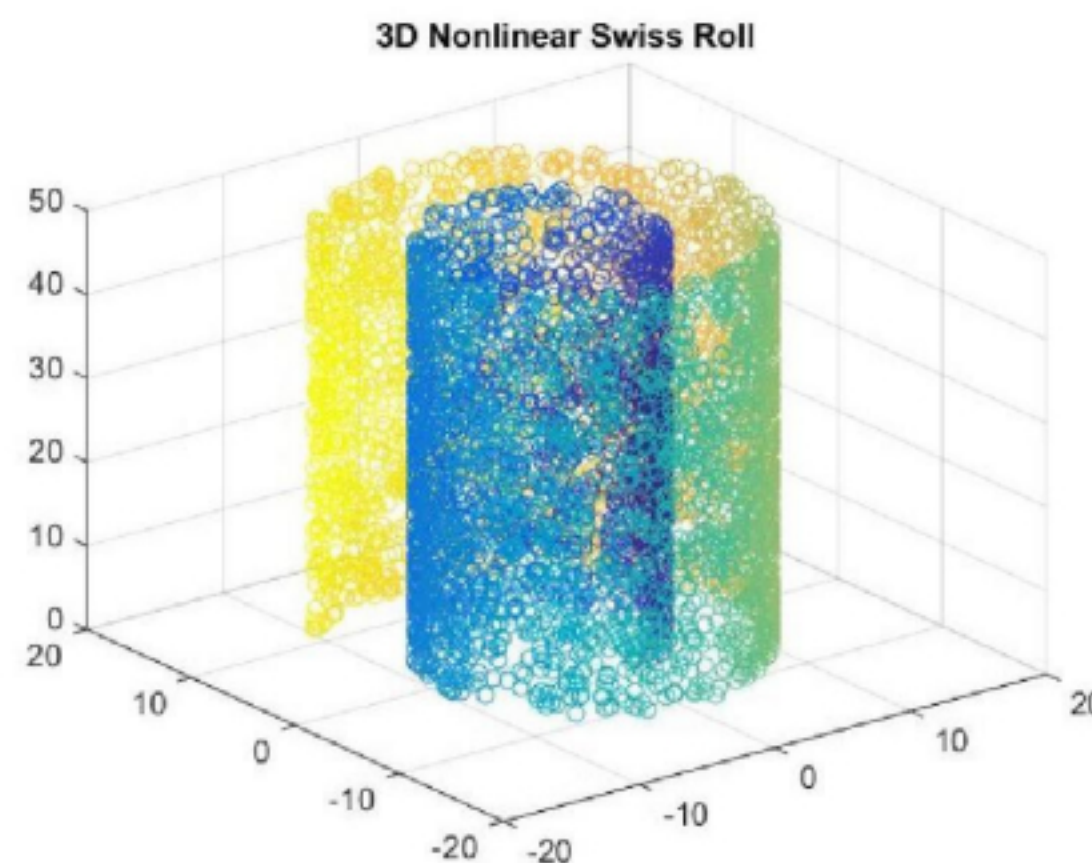
# Dimensionality reduction with autoencoders

## Learning nonlinear manifolds, dimensionality reduction

Linear vs nonlinear dimensionality reduction



- Key assumption: there is low-dimensional structure of the data
- Balance in sensitivity to data between building accurate reconstructions and preventing from memorizing (overfitting)





# Generating samples from autoencoders

- Unsupervised learning
  - no labels
  - the objective is to capture the hidden structure in the data
  - density estimation, clustering, feature learning, dimensionality reduction
- Generative approach allows even for generating samples
  - probabilistic in nature: learning **the joint  $P(x, y)$**  -> ambitious
  - training data  $\sim P_{data}(x)$  -> generated samples  $\sim P_{model}(x)$
  - capable for uncovering underlying latent variables
  - could be used for many purposes, e.g. debiasing or outlier detection
  - some flagship examples of deep latent variable models: DBN, GAN, VAE



# Overcomplete autoencoders

## *Overcomplete* regularised autoencoders

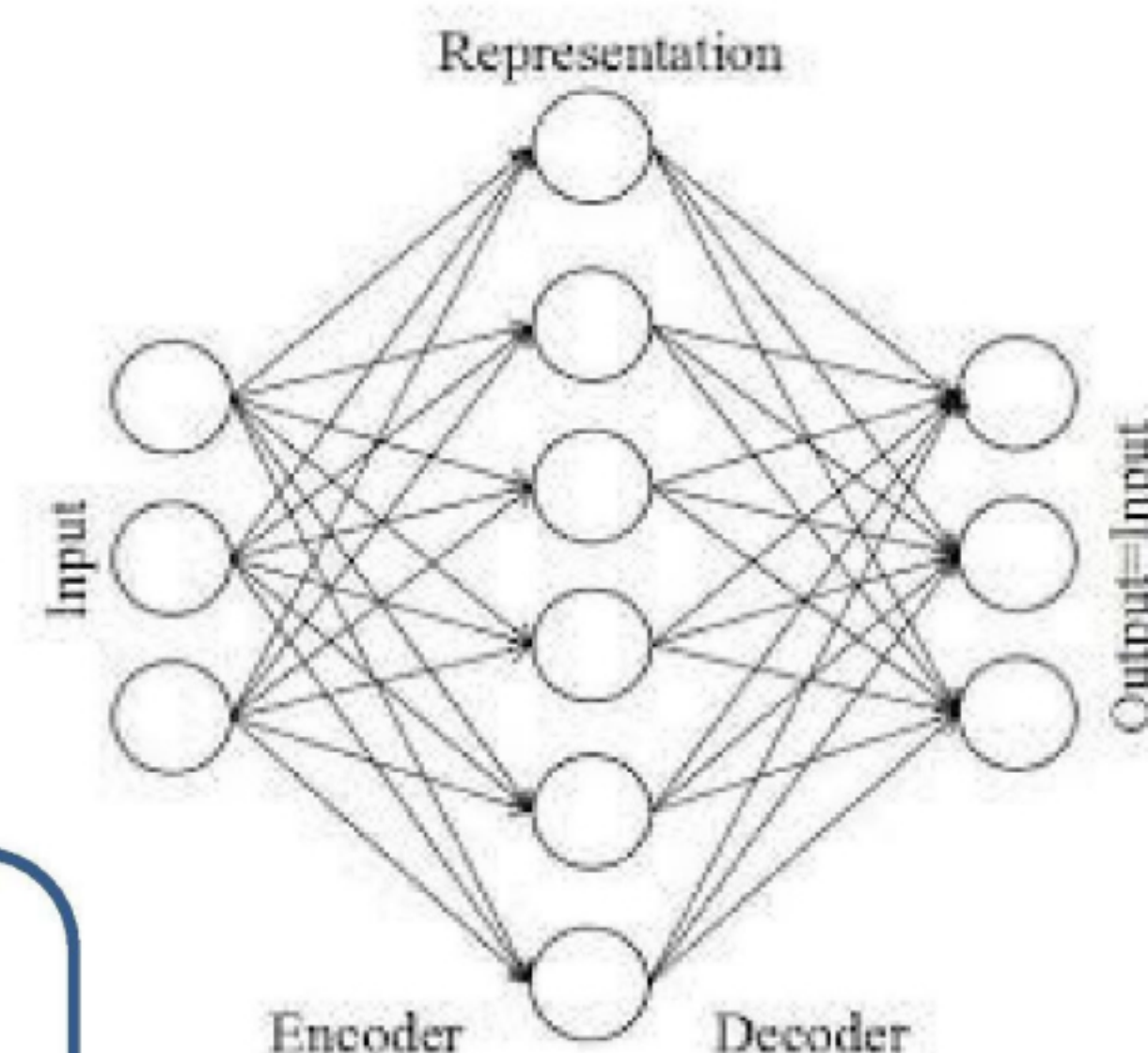
- larger hidden layer size than that of the input and output
- the need for explicit regularisation (to avoid overfitting - copying input to the output):

*Reconstruction* loss + *Regularisation penalty*

$$L(x, d(h)) + \Omega(h), \quad h = e(x)$$

When training autoencoders there is a **compromise**

- I. Need to approximately recover  $x$  – *reconstruction* force
- II. Need to satisfy the regularization term – *regularisation* force.



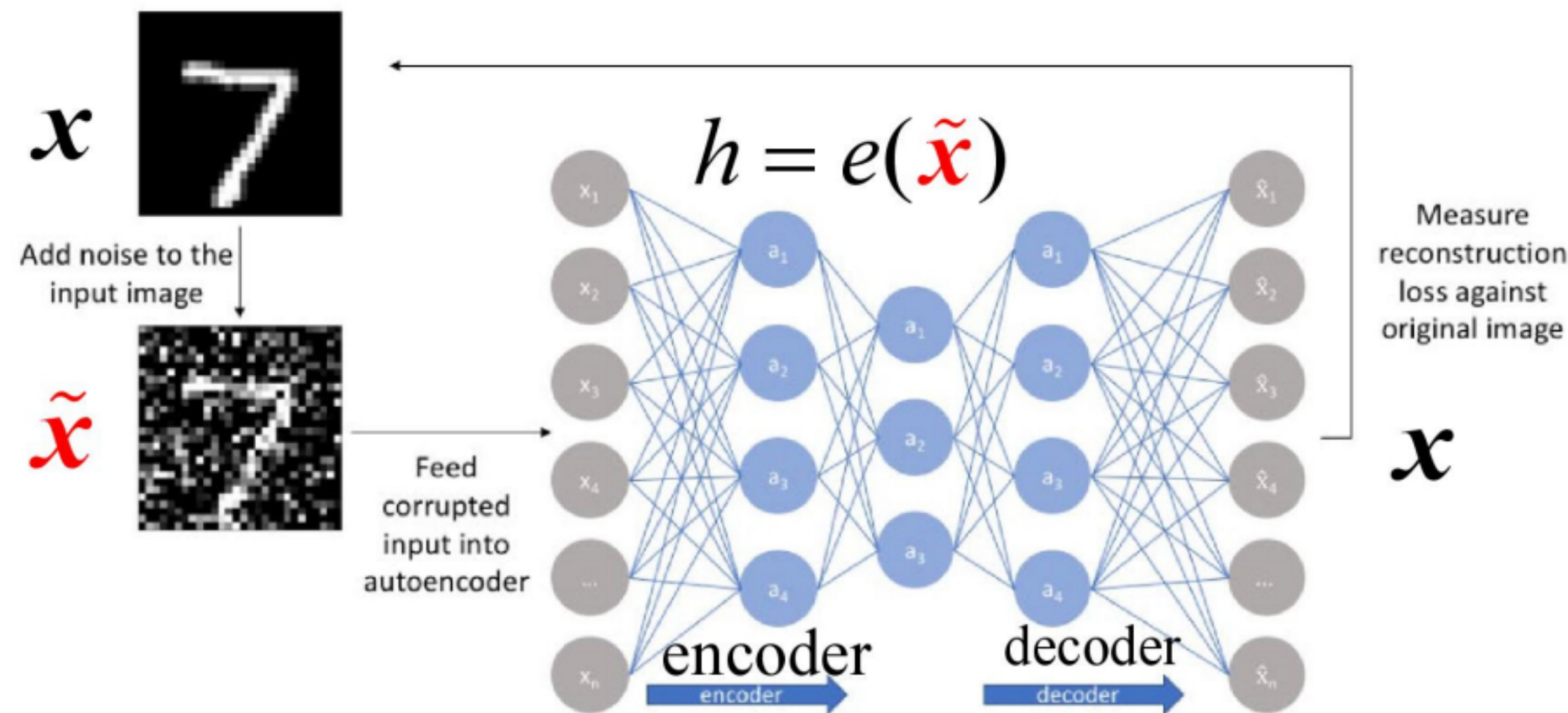


# Denoising autoencoders

$$L(\mathbf{x}, d(e(\tilde{\mathbf{x}}))), \quad h = e(\tilde{\mathbf{x}})$$

Corrupted copy of  $\mathbf{x}$

Autoencoders have to undo this corruption beyond simply coping the input.





# Denoising autoencoders

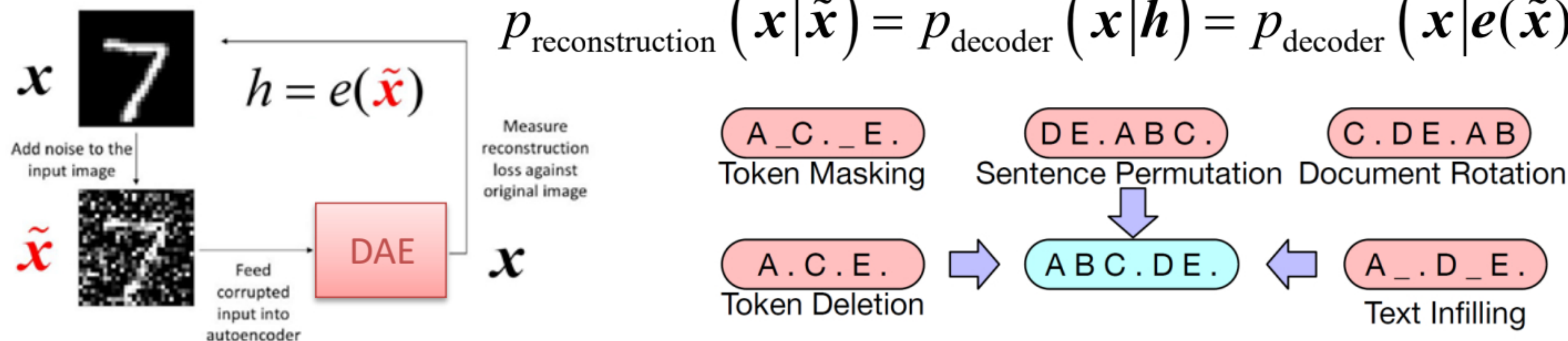
$$L(\mathbf{x}, d(e(\tilde{\mathbf{x}}))), \quad h = e(\tilde{\mathbf{x}})$$

Corrupted copy of  $\mathbf{x}$

Autoencoders have to undo this corruption beyond simply coping the input.

1. A training sample is sampled from the training data.
2. A corrupted version of the sample  $\mathbf{x}$  is drawn from some noise model  $\mathcal{M}_d(\tilde{\mathbf{x}}|\mathbf{x})$ .
3. Pairs are used as a training sample to estimate the reconstruction distribution:

$$p_{\text{reconstruction}}(\mathbf{x}|\tilde{\mathbf{x}}) = p_{\text{decoder}}(\mathbf{x}|\mathbf{h}) = p_{\text{decoder}}(\mathbf{x}|e(\tilde{\mathbf{x}}))$$





# Denoising autoencoders

Original text

Thank you ~~for inviting~~ me to your party ~~last~~ week.

Inputs

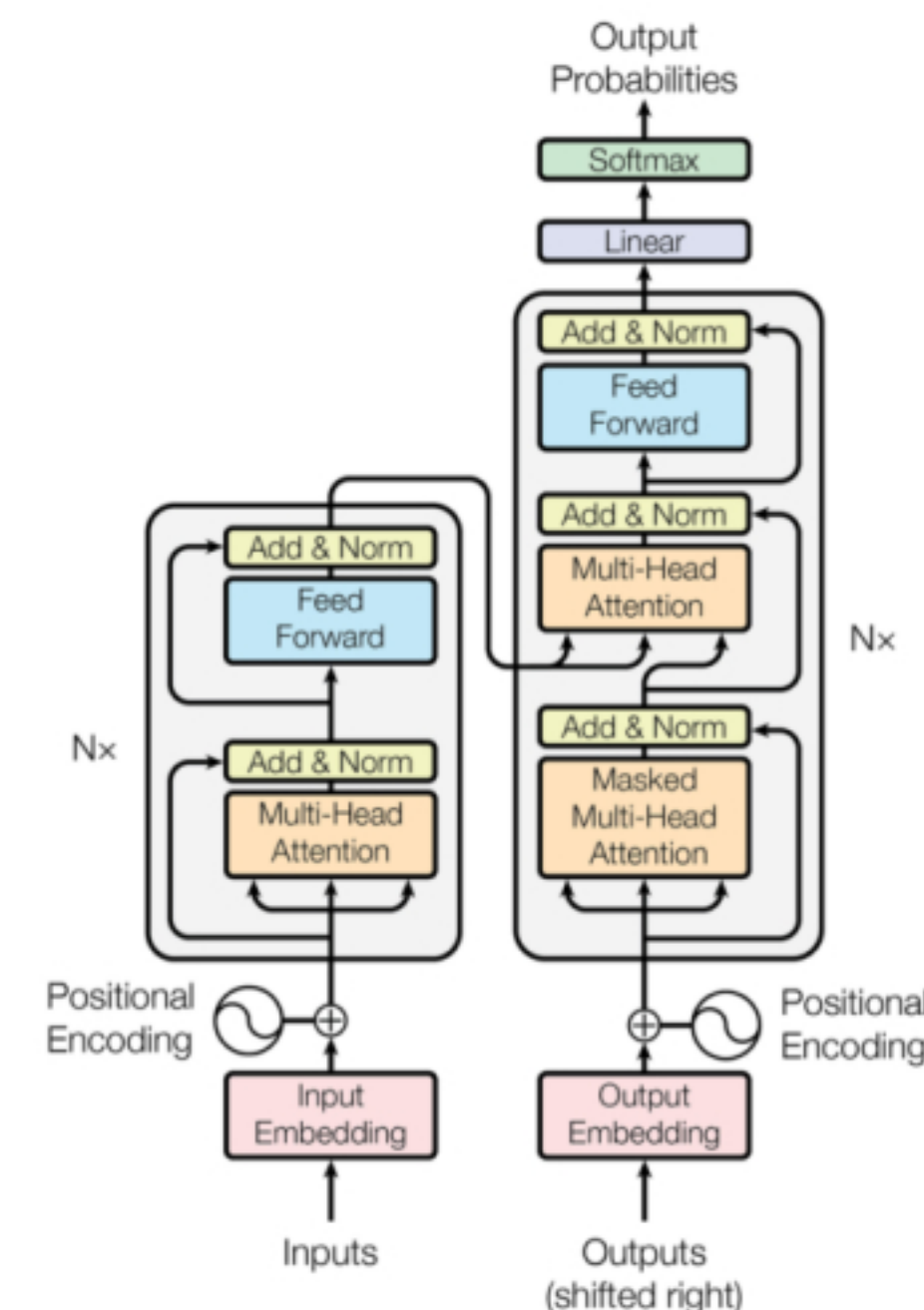
Thank you <X> me to your party <Y> week.

Text after masking.

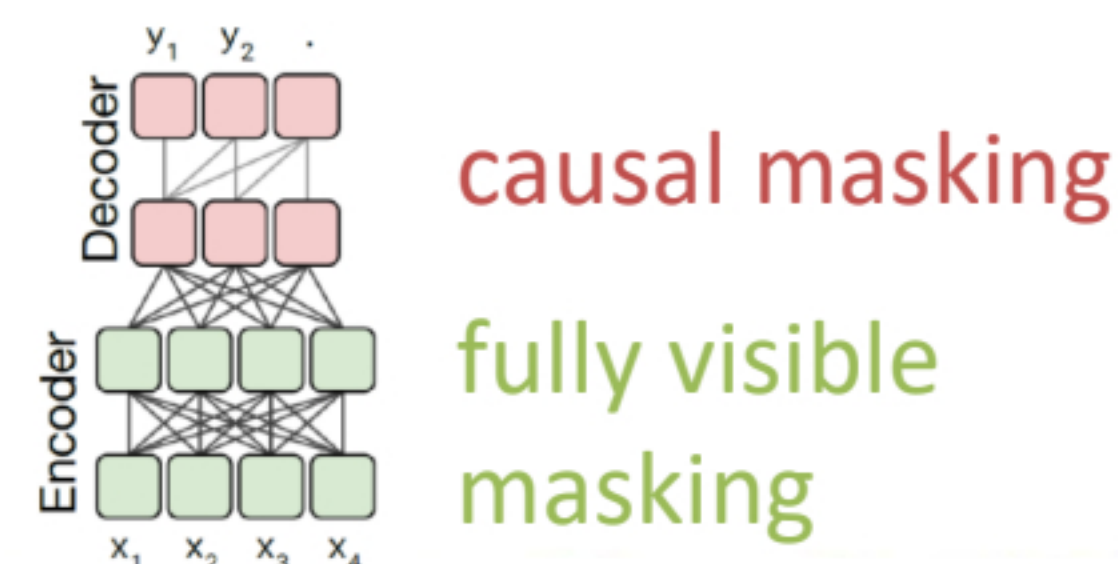
Targets

<X> for inviting <Y> last <Z> Expected guesses (missing words).

- T5 is pre-trained on a masked language modeling “span-corruption” objective (masking process).
- Consecutive spans of input tokens are replaced with a [MASK] token and the model is trained to reconstruct the masked-out tokens.
- The use of a unified “text-to-text” format allows reuse exactly the same training objective (i.e. teacher-forced maximum likelihood) for every task while a single set of hyperparameters  $\vec{w}$  is used for effective fine-tuning on any downstream task.

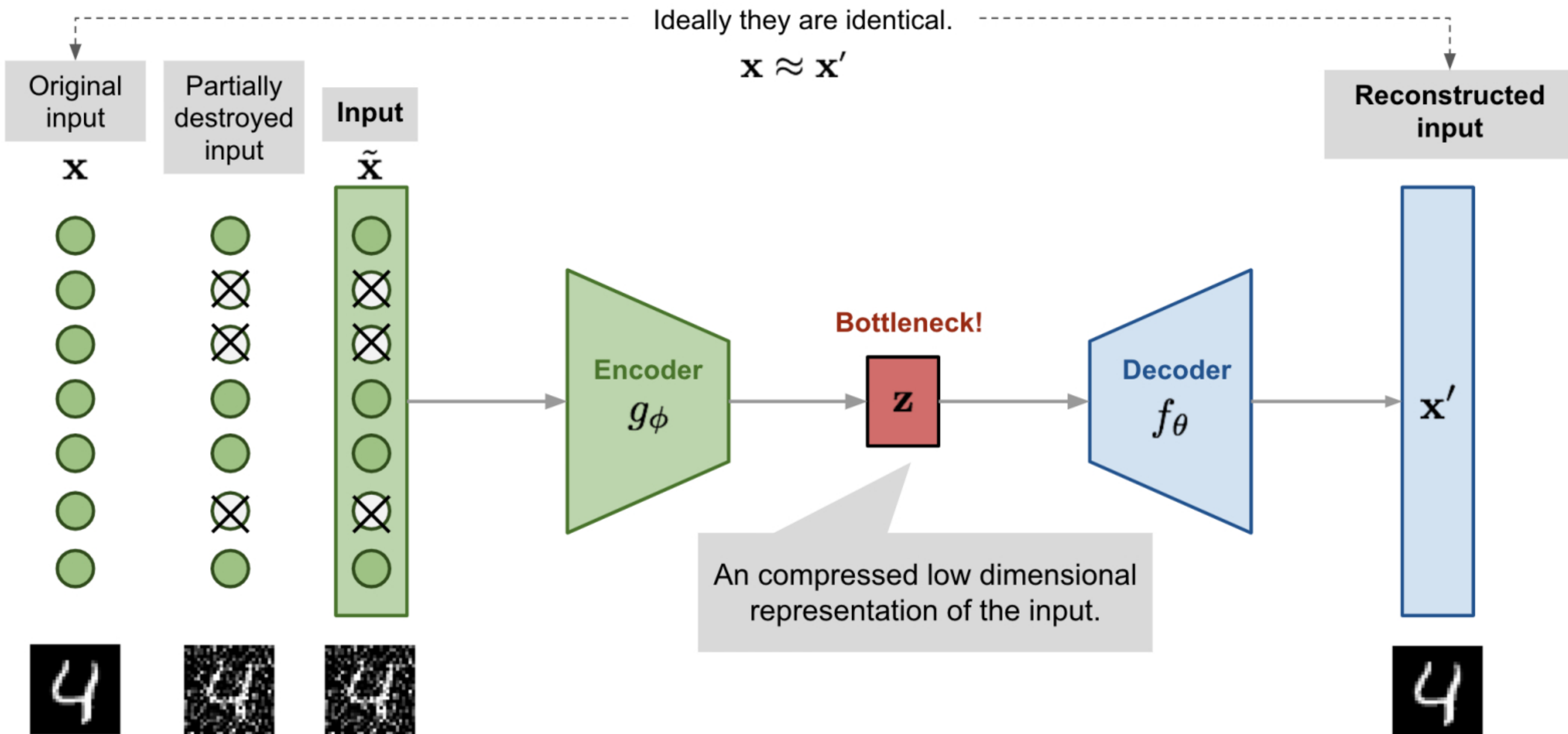


The transformer –  
Typical Encoder-Decoder  
architecture



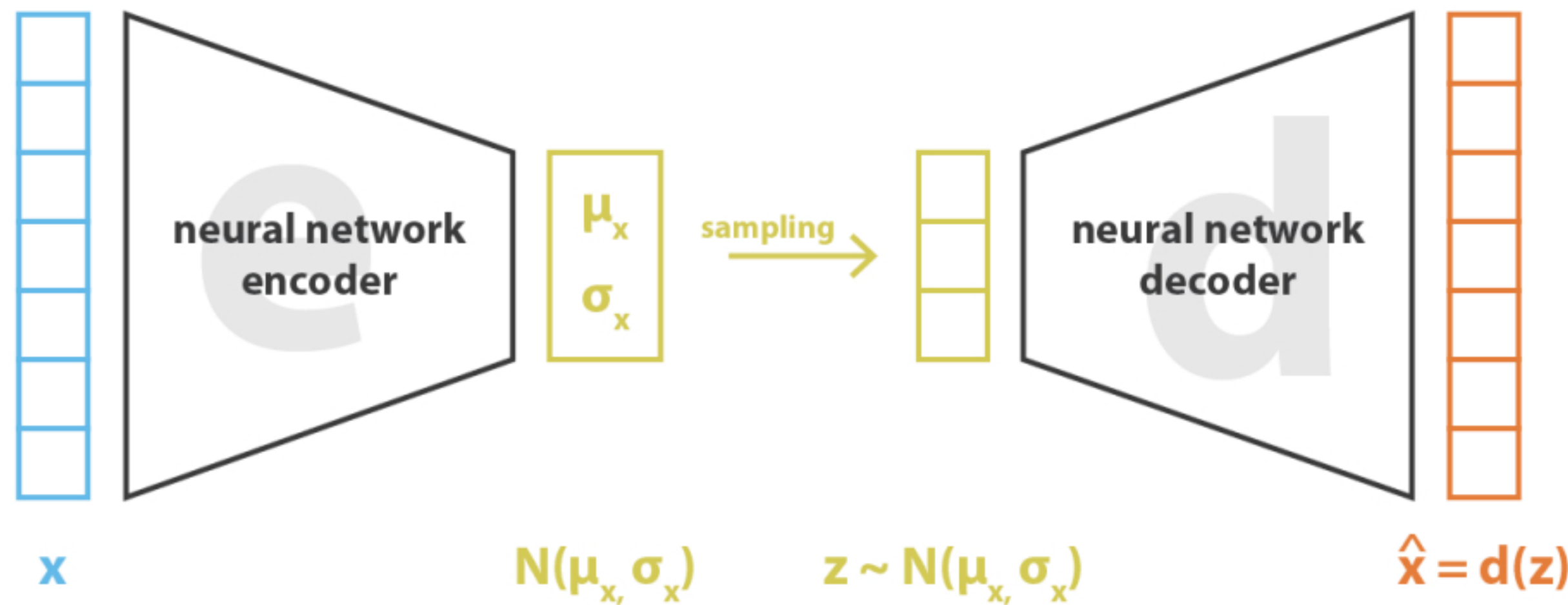
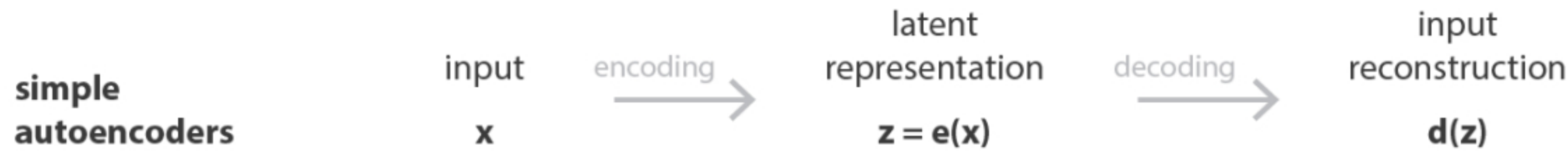


# Denoising autoencoders





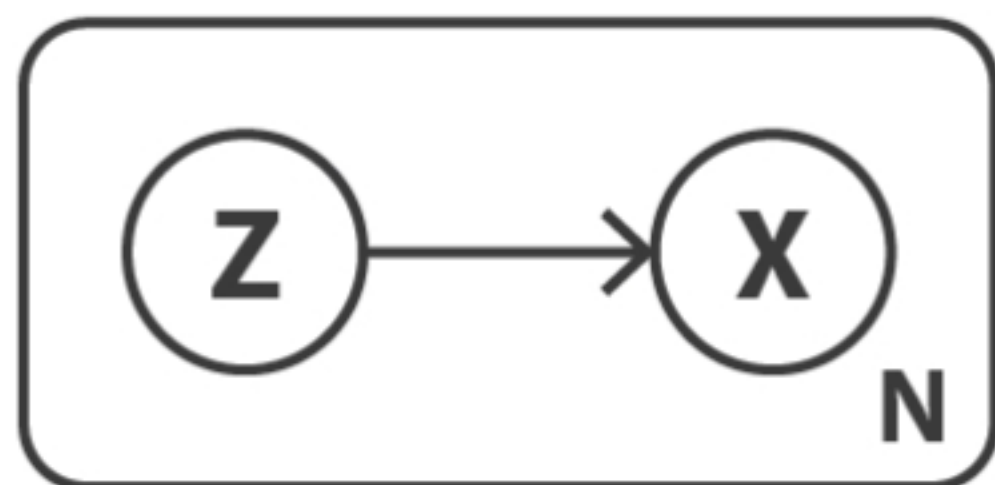
# Variational autoencoders



variational approach  
with some  $q(\mathbf{z}|\mathbf{x})$

$$\text{KL} (p^* || q) = \sum_{\mathbf{x}} p^*(\mathbf{x}) \log \frac{p^*(\mathbf{x})}{q(\mathbf{x})}$$

# Variational autoencoders



$$p(z|x) = \frac{p(x|z)p(z)}{p(x)} = \frac{p(x|z)p(z)}{\int p(x|u)p(u)du}$$

**intractable**

$$p(z) \equiv \mathcal{N}(0, I)$$

$$p(x|z) \equiv \mathcal{N}(f(z), cI) \quad f \in F \quad c > 0$$

$$q_x(z) \equiv \mathcal{N}(g(x), h(x)) \quad g \in G \quad h \in H$$

We need to resort to a variational approach with some  $q(z|x)$  and minimise KL divergence:

$$\min \text{KL}(q(z|x) \| p(z|x))$$



$$\max E_{q(z|x)} \log p(x|z) - \text{KL}(q(z|x) \| p(z)) \leq \log p(x)$$

*Lower bound* for  
the likelihood



Loss function:  $\min L(x, \tilde{x}) + \sum_j \text{KL}(q_j(z_j|x) \| p(z_j))$

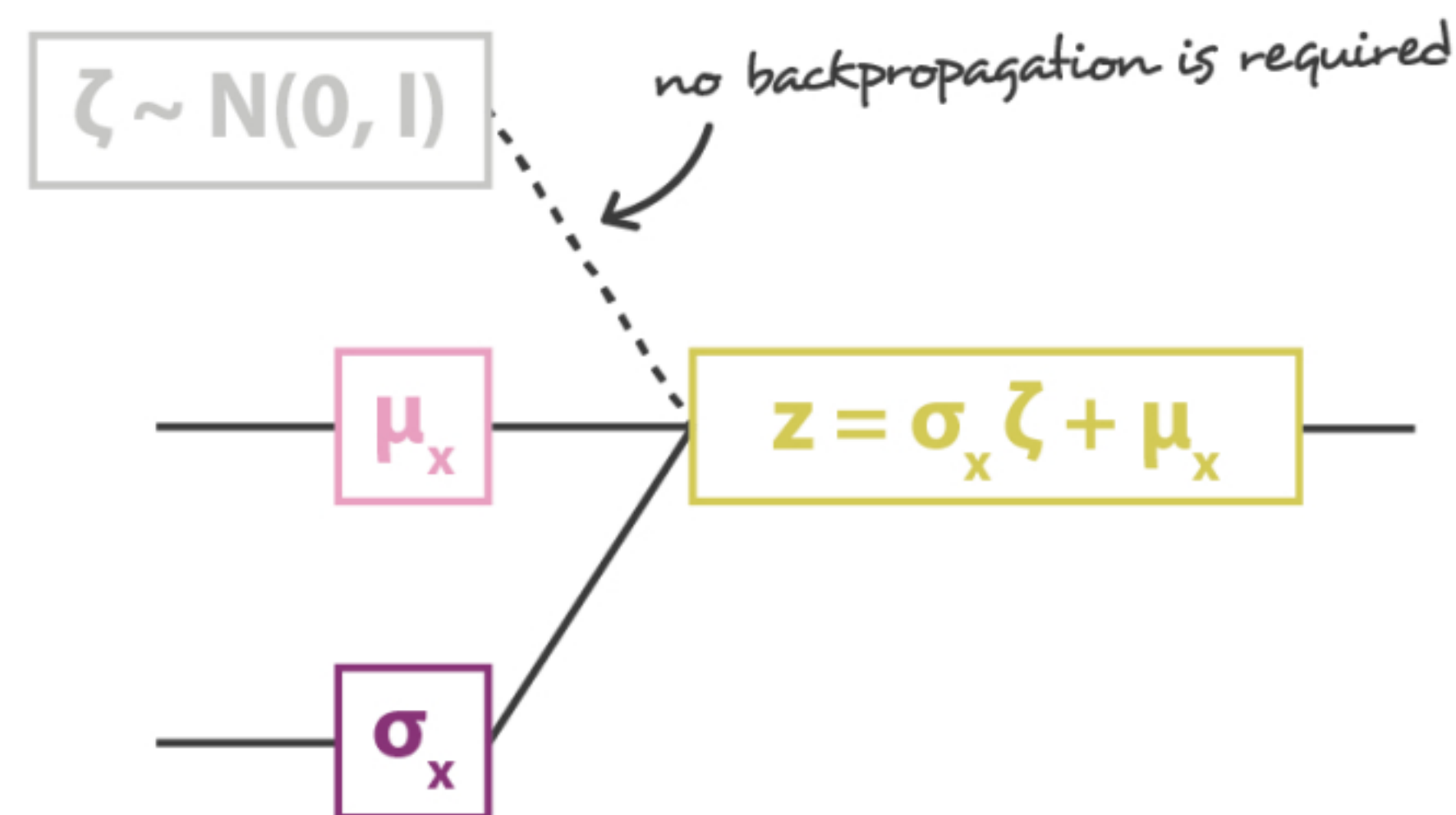
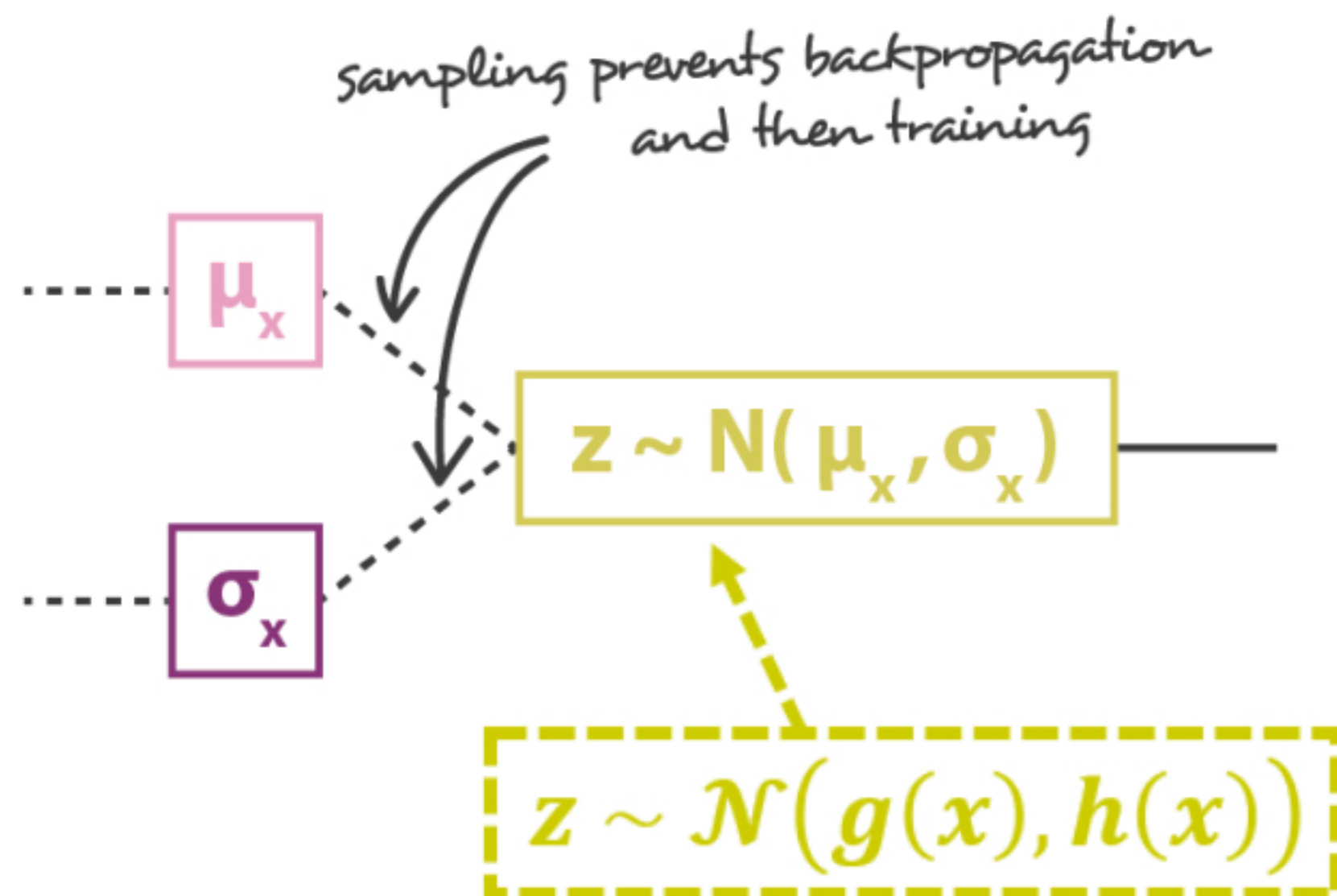
*regularisation*



# Variational autoencoders

— no problem for backpropagation

----- backpropagation is not possible due to sampling

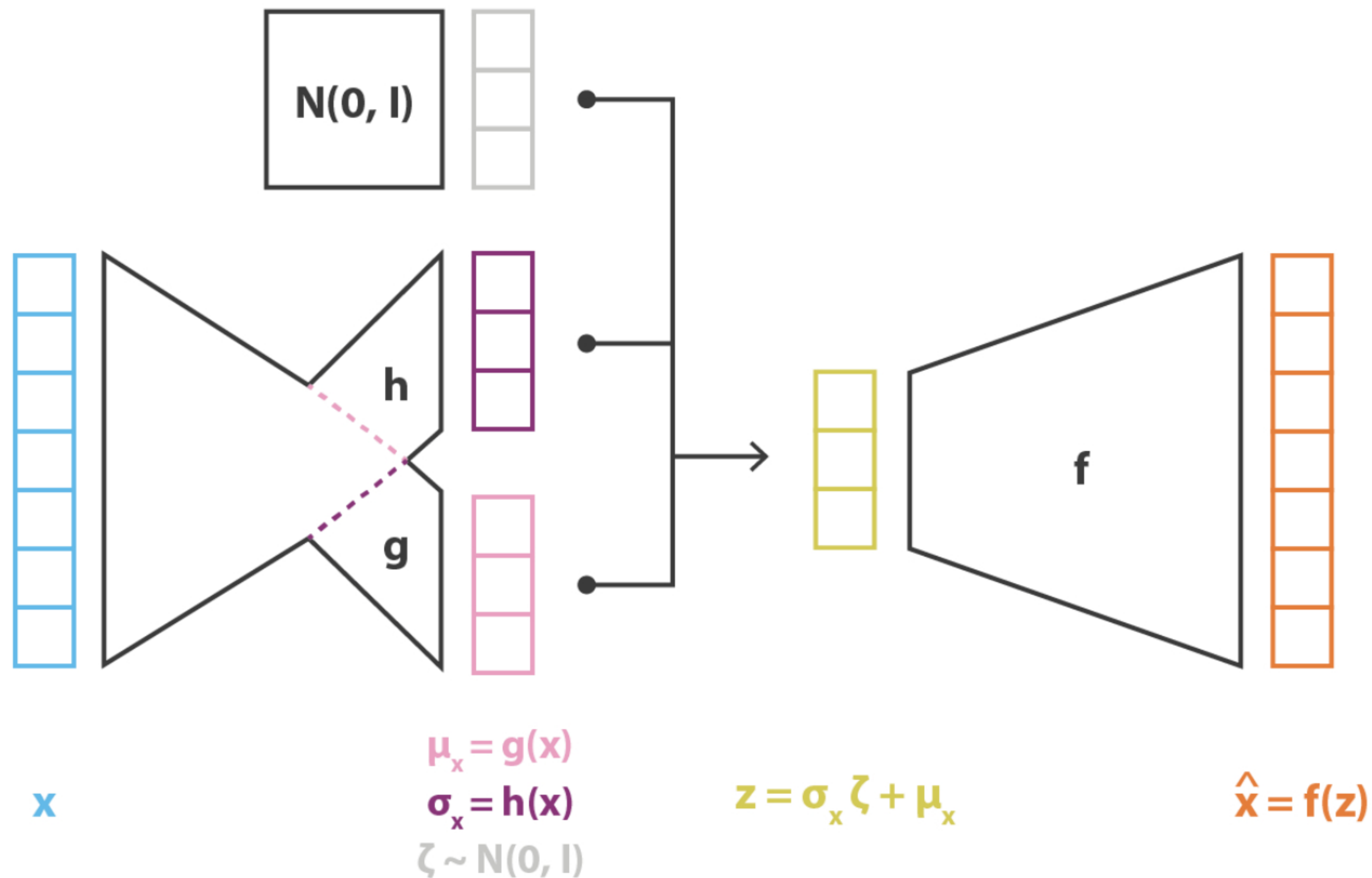


A simple trick, called **reparametrisation trick**, is used to make the gradient descent possible despite the random sampling that occurs halfway of the architecture

Loss function:  $\min L(\mathbf{x}, \tilde{\mathbf{x}}) + \sum_j^{|z|} \text{KL}(q_j(z_j | \mathbf{x}) \| p(z_j))$

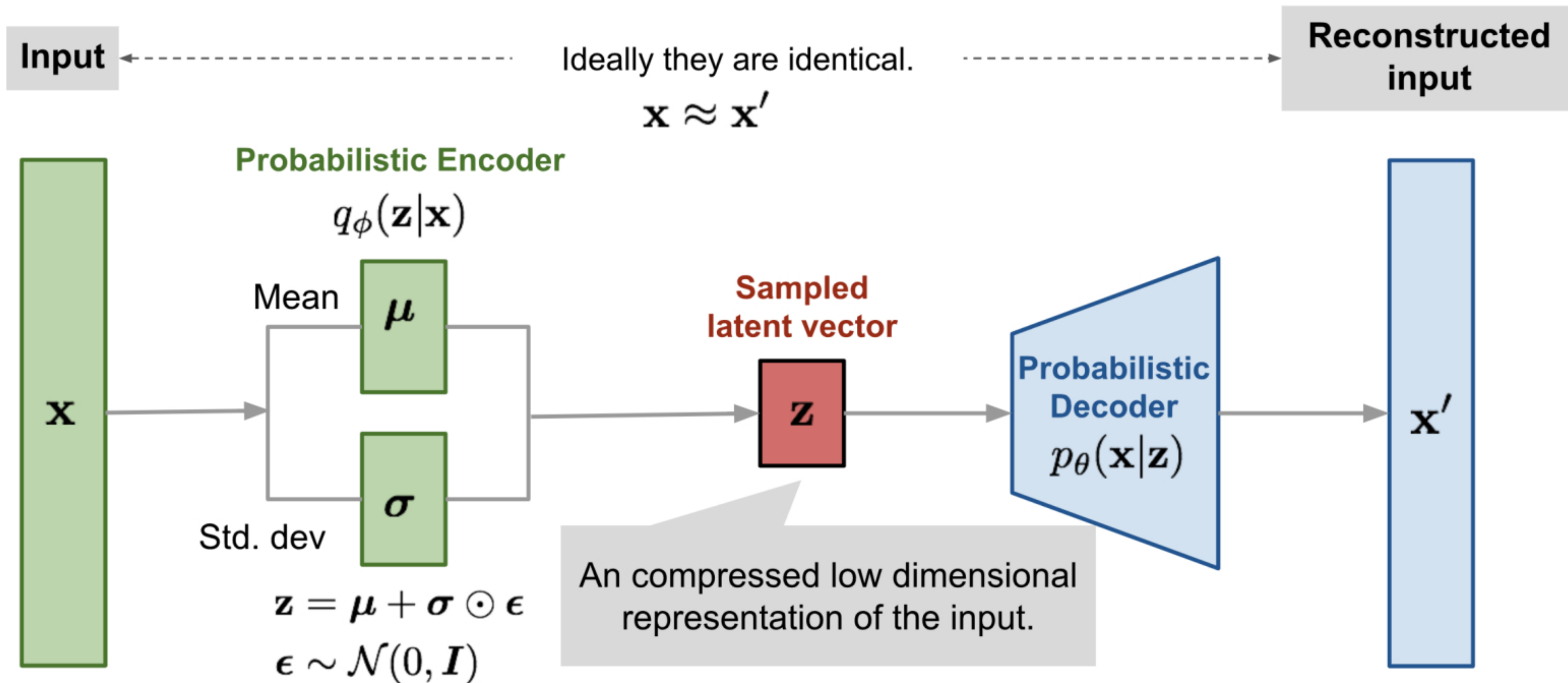
regularization can also help to get good properties

# Variational autoencoders



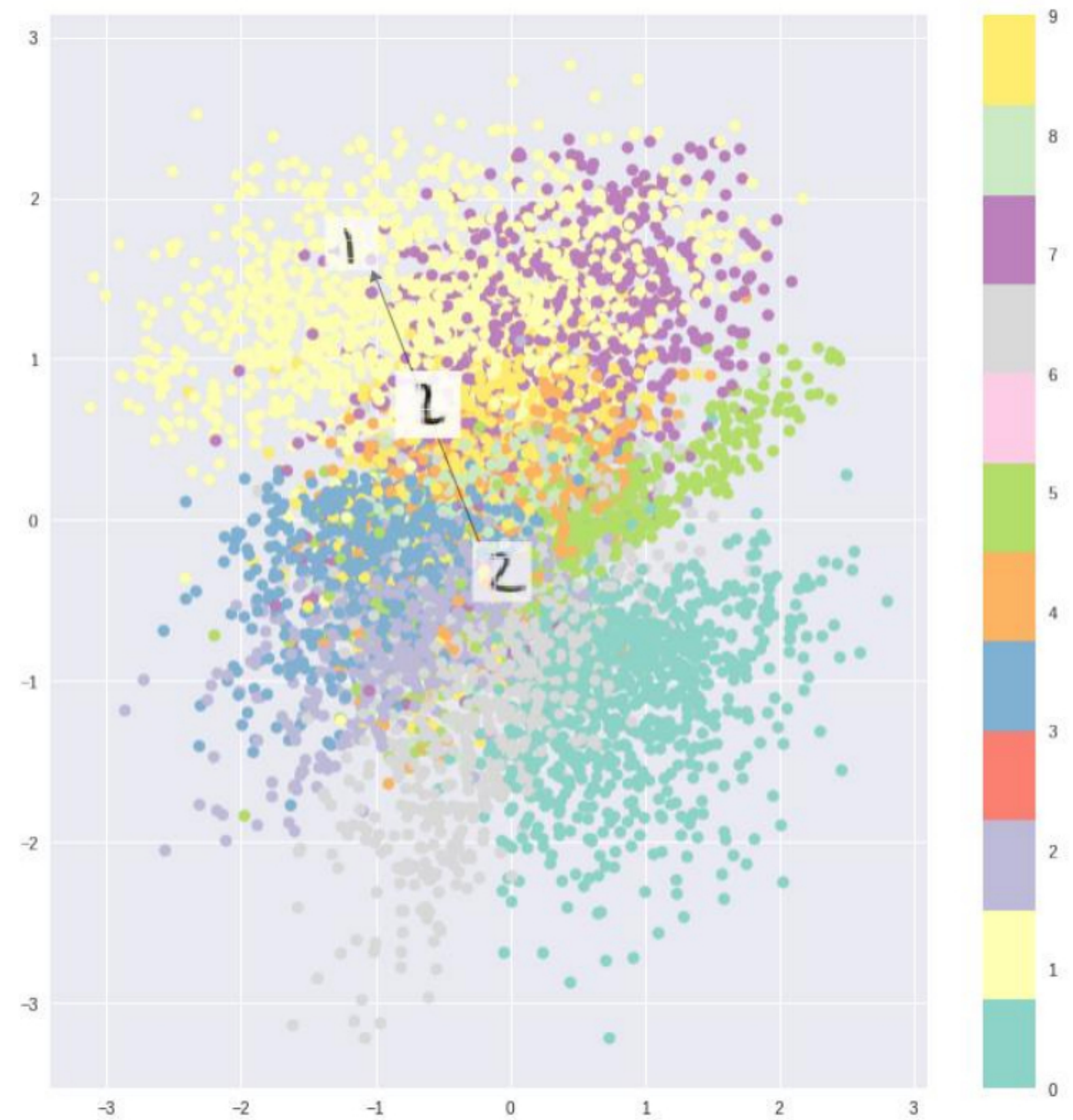
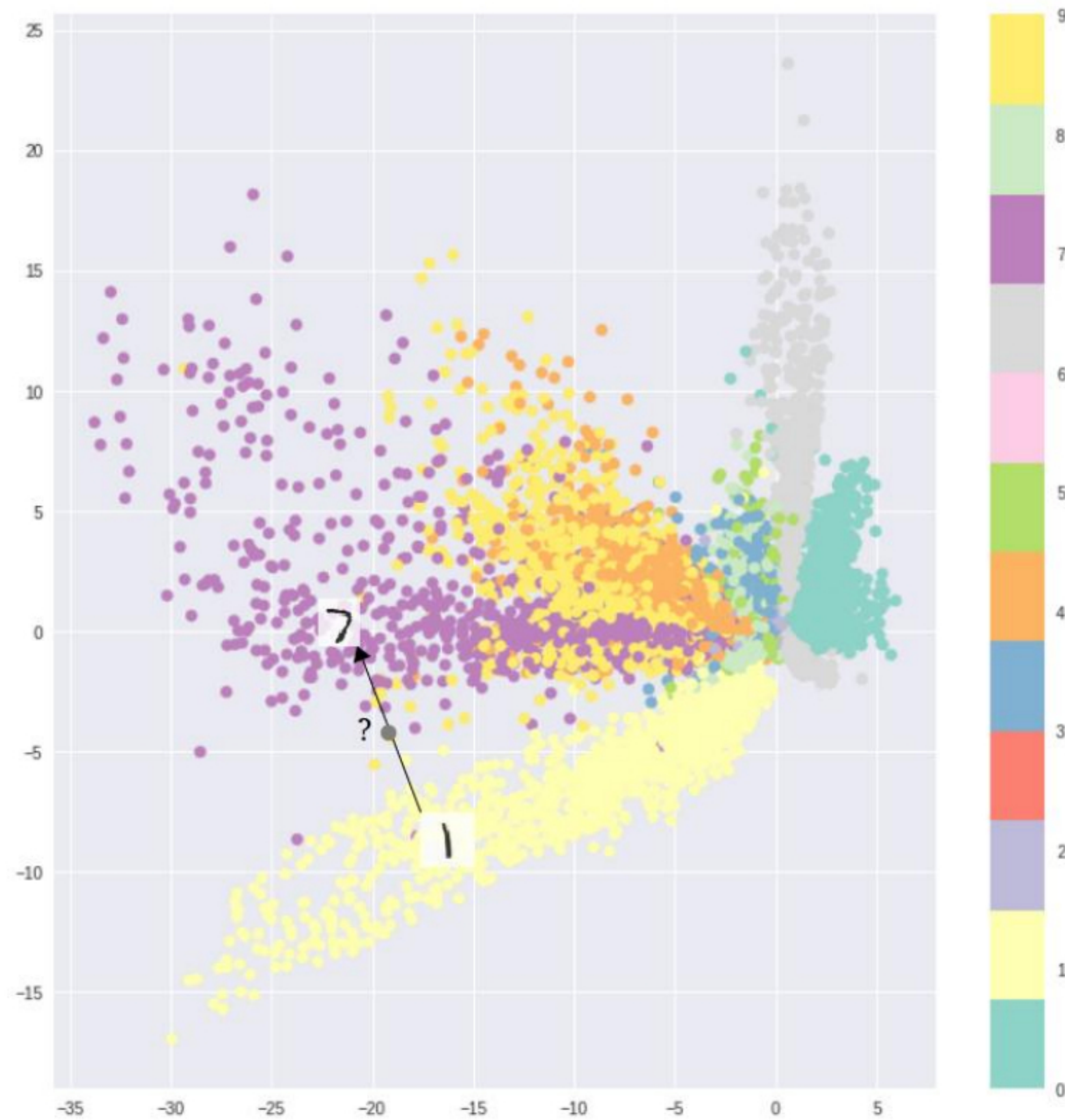


# Variational autoencoders



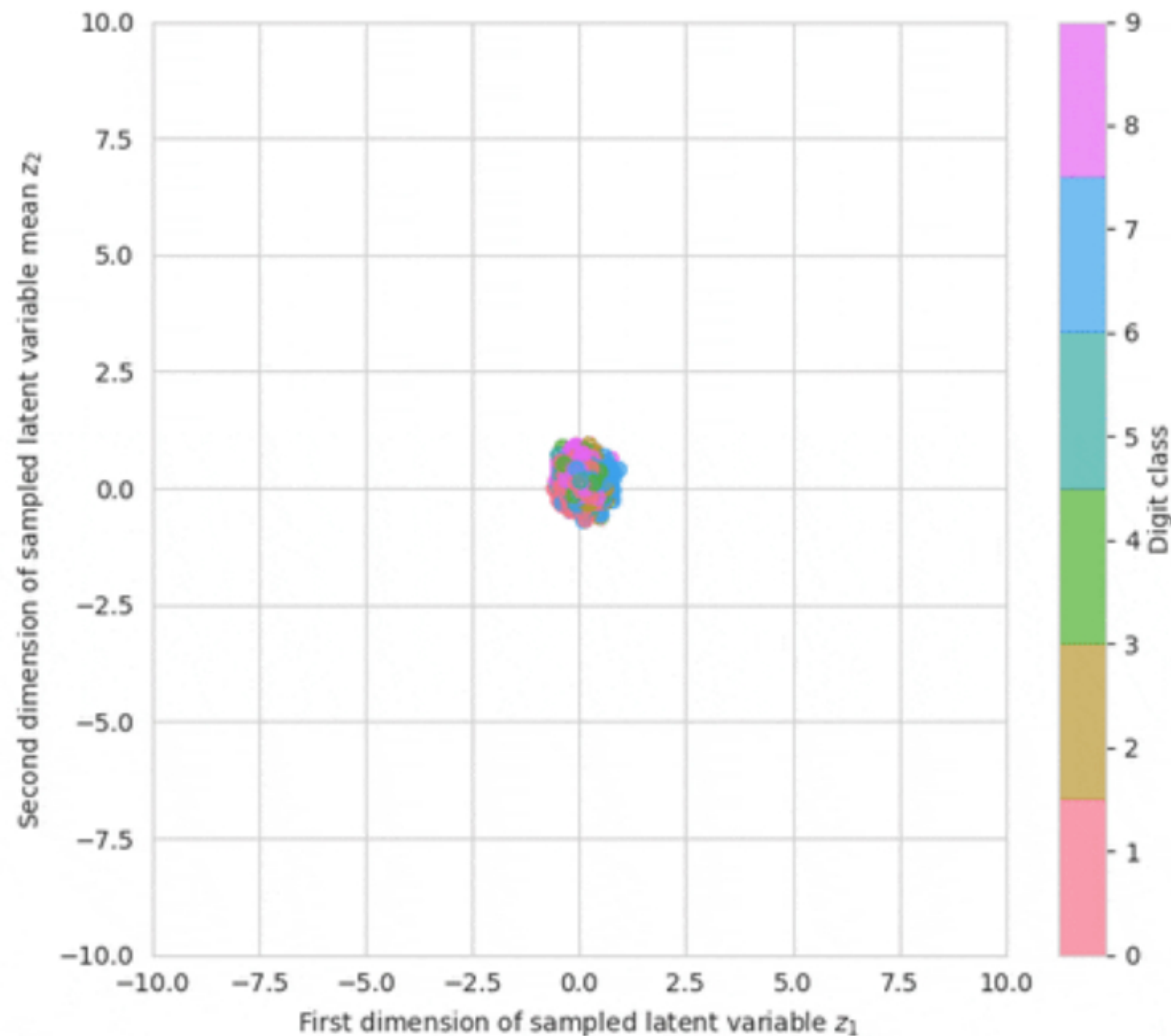


# Variational autoencoders





# Variational autoencoders



again, there is a compromise between the *reconstruction* and the *regularization* force

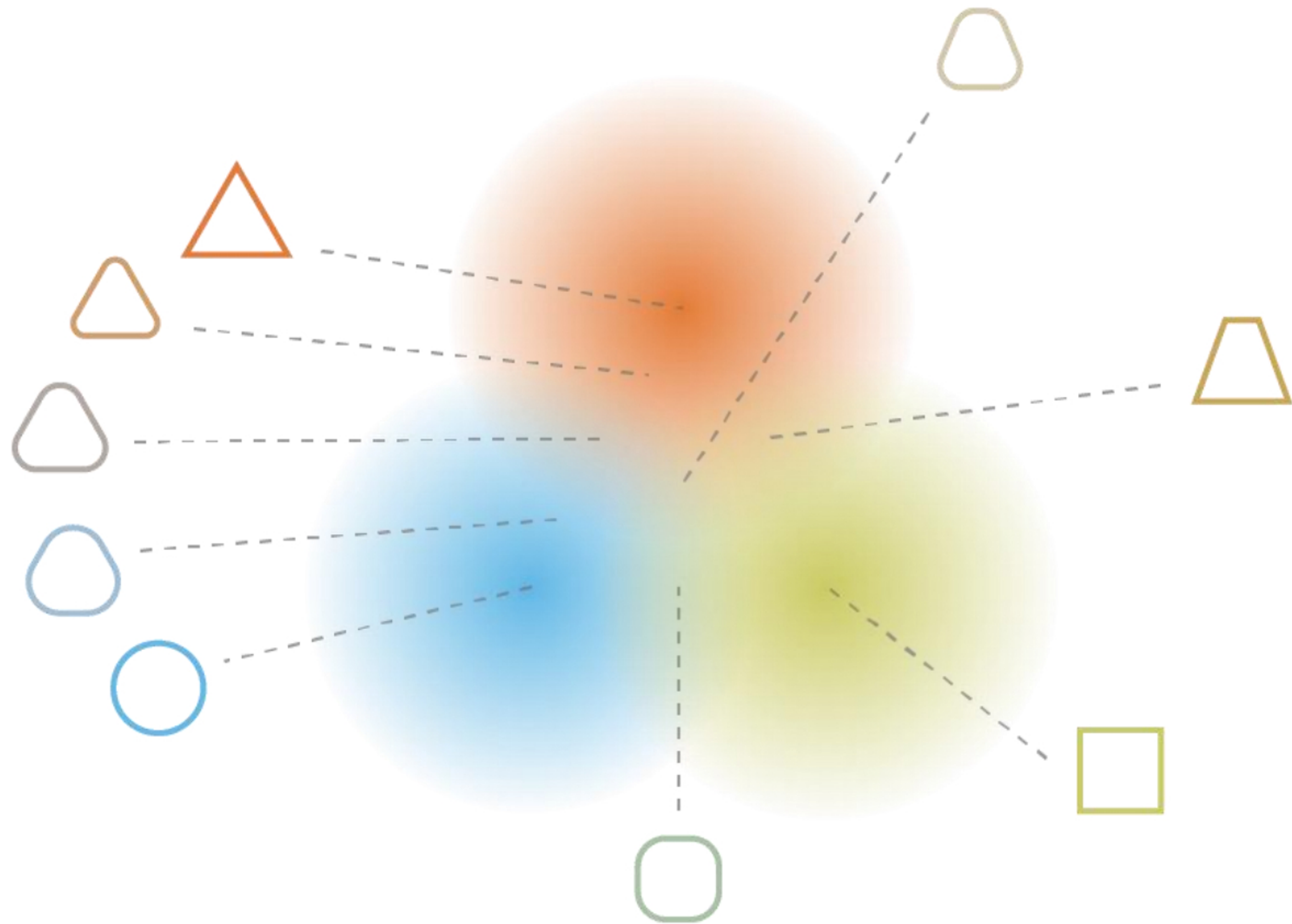


“bad” latent space  
distinct and isolated  
islands in the latent space

“good” properties  
of the latent space

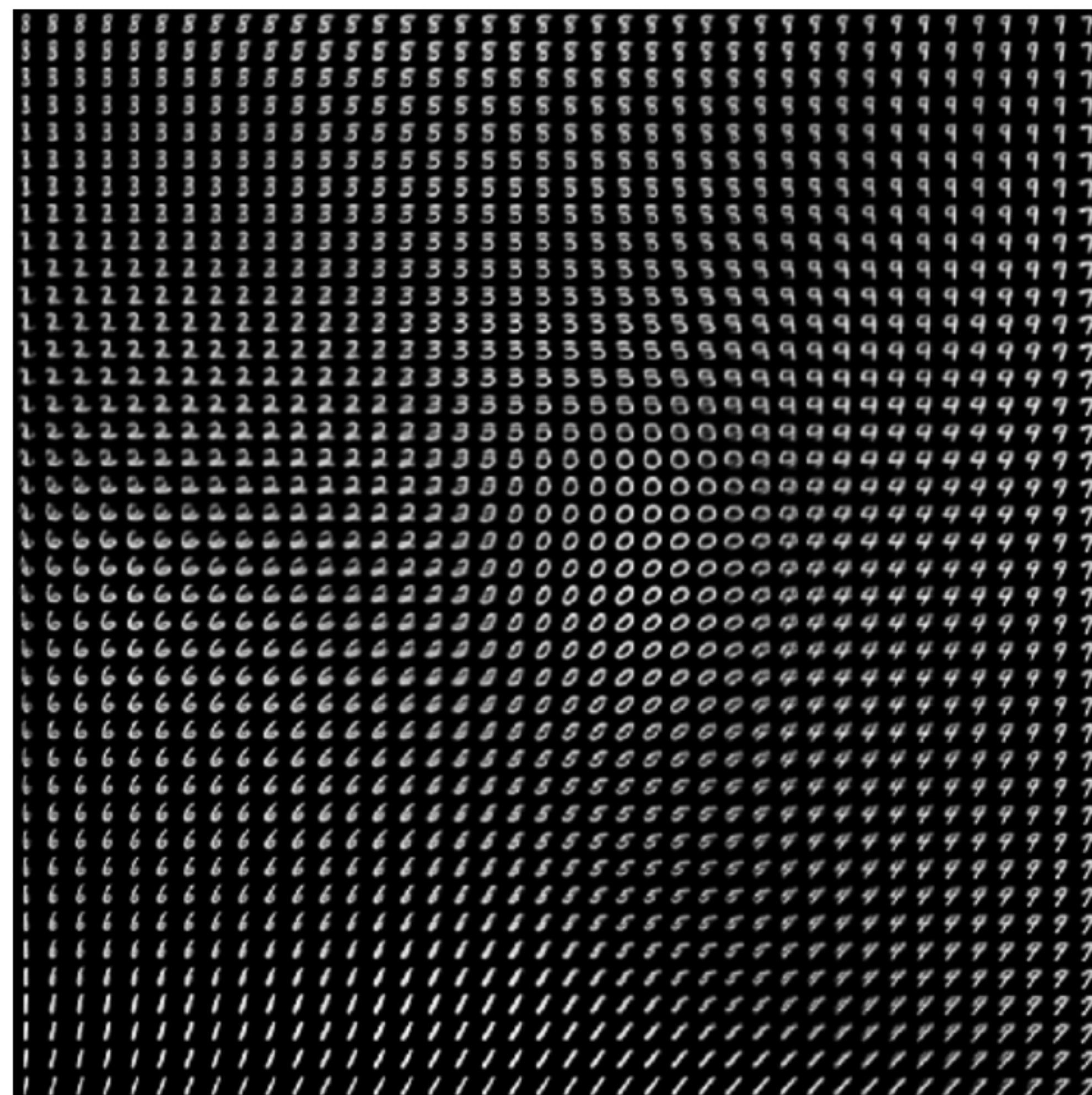
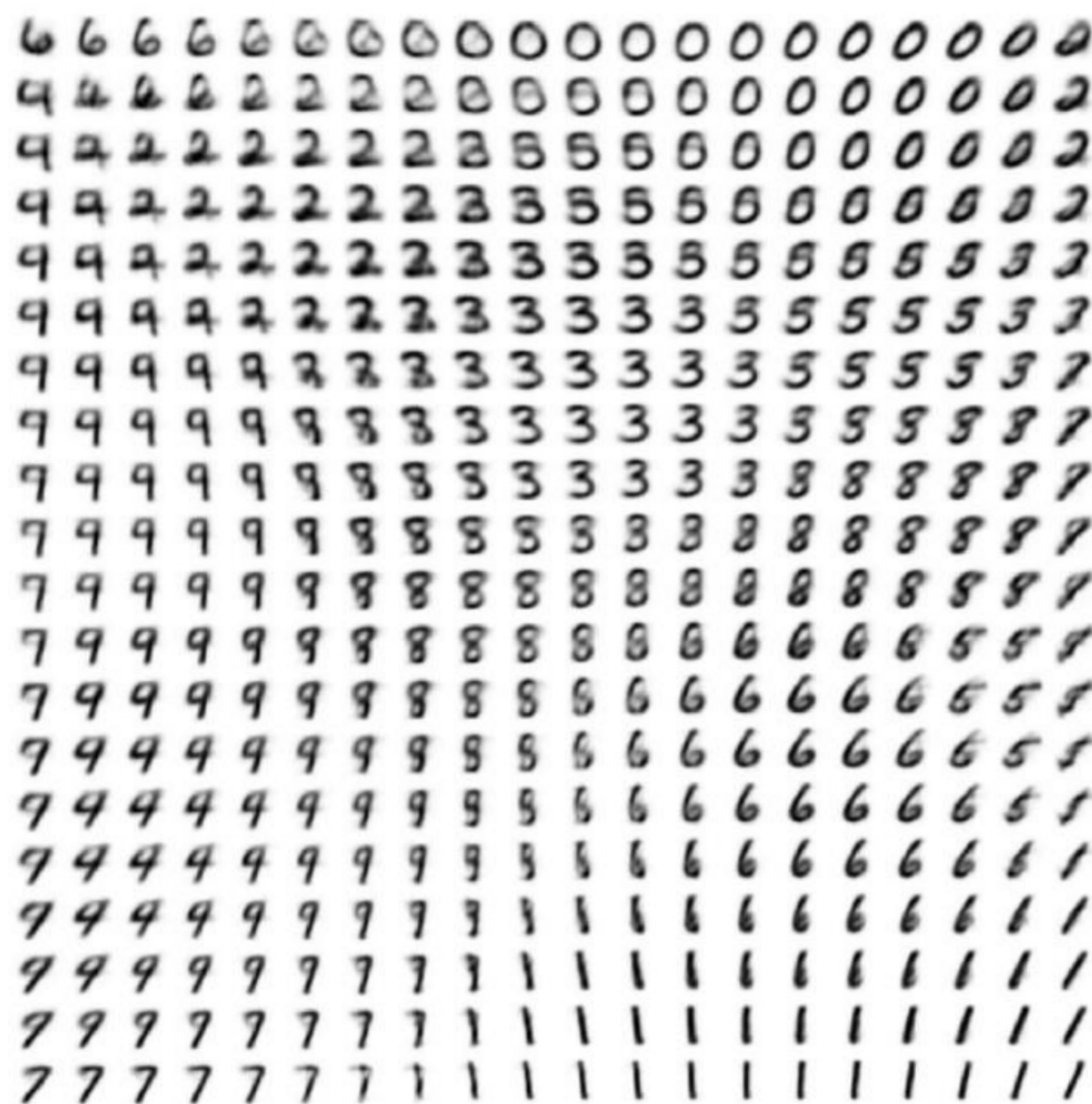
regularization can also help  
to get good properties

# Variational autoencoders





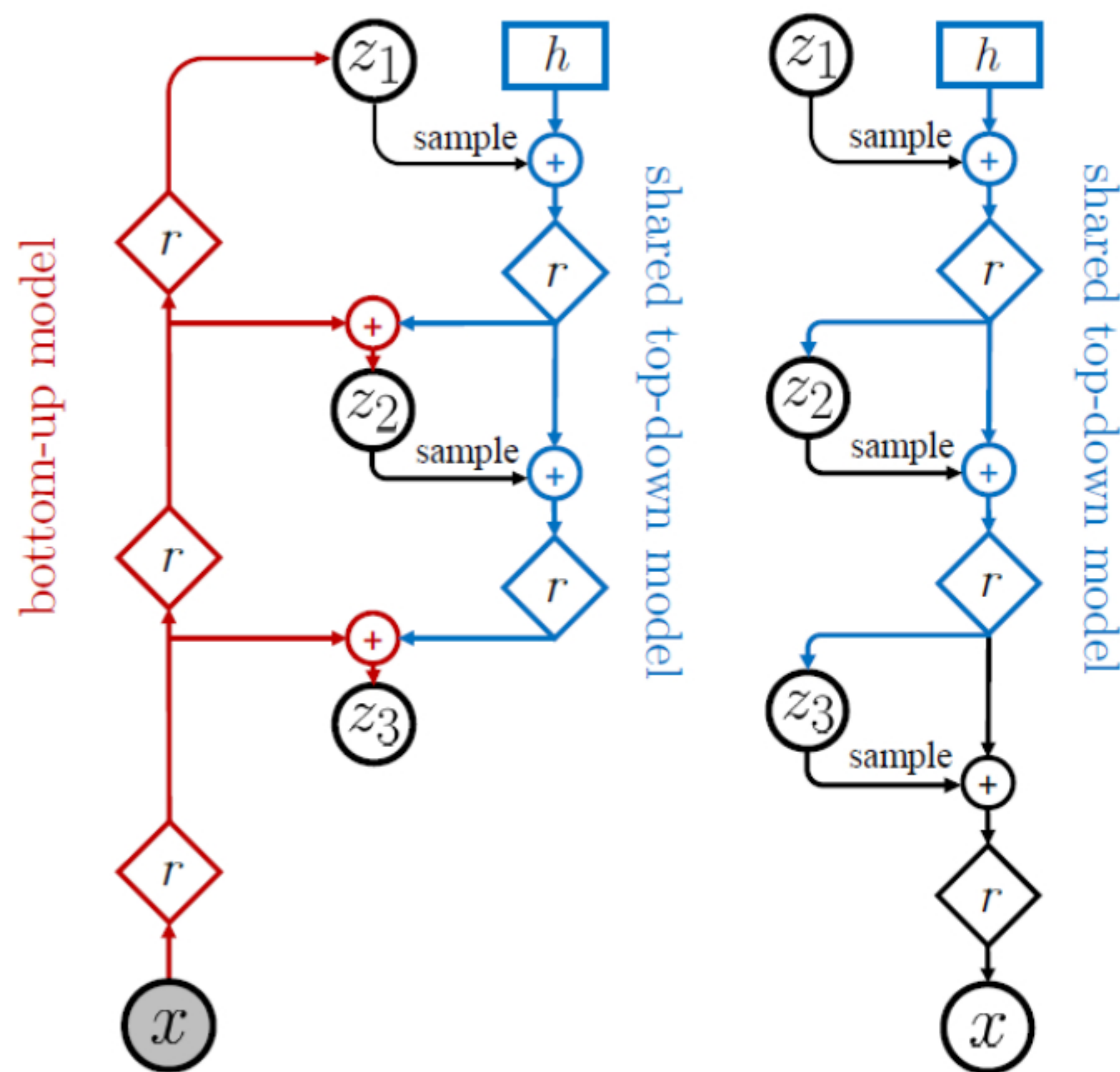
# Variational autoencoders





# Variational autoencoders

## Hierarchical Variational Autoencoders:



(a) Bidirectional Encoder (b) Generative Model  
[Vahdat and Kautz "NVAE: A Deep Hierarchical Variational Autoencoder" NeurIPS 2020]

- Training generative models in the form of:
$$p(\mathbf{x}, \mathbf{z}) = p(\mathbf{z})p(\mathbf{x}|\mathbf{z})$$
- Since we don't know the true posterior  $p(\mathbf{z}|\mathbf{x})$  we may use an approximate posterior of the encoder  $q(\mathbf{z}|\mathbf{x})$ , do deal with the posterior being intractable.
- To increase expressiveness of both  $q(\mathbf{z}|\mathbf{x})$  and prior  $p(\mathbf{z})$  we partition the latent factor  $\mathbf{z}$  into disjoint groups  $\mathbf{z} = \{z_1, z_2, \dots, z_L\}$ .
- By this twist we get:  $p(\mathbf{z}) = \prod_{\ell} p(z_{\ell} | \mathbf{z}_{<\ell})$ 
$$q(\mathbf{z}|\mathbf{x}) = \prod_{\ell} q(z_{\ell} | \mathbf{z}_{<\ell}, \mathbf{x})$$

We train the model w.r.t.  $\mathcal{L}_{\text{VAE}}(\mathbf{x})$  using the reparameterization trick. See their paper for more details.



# Summary of Deep Generative Models

- Generative modelling aims at modelling data distribution (probabilistic approach)
  - learning **the joint  $P(x, y)$**  -> ambitious (if labels exist)
  - rapidly growing interest in unsupervised learning (no labels)
  - scope for generating new data samples: training data  $\sim P_{data}(x)$  -> generated samples  $\sim P_{model}(x)$
  - capable for uncovering underlying latent variables (learning represent.)
  - some flagship examples of deep latent variable models: DBN, GAN, VAE
- Traditionally, DBNs have been used to model joint distributions
  - greedy layer-wise pretraining of the latent (hidden) representations
  - often acting as a hybrid model – discriminative and generative



# Summary of Deep Generative Models

- Generative Variational Autoencoders (VAE) balance the reconstruction accuracy and “desirable/good” properties in the latent space
  - the latent space irregularity is tackled by
    - the encoder returning a distribution over the latent space instead of a single point (a probabilistic “twist” on AE -> stochastic behaviour)
    - the loss function incorporating a regularisation term over the returned latent distribution to ensure a good organisation of the latent space
  - VAE allows for defining a meaningful generative process using well-organised latent space, which enables sampling from the distribution as well as altering and exploring variations on the existing data
  - VAEs have a probabilistic interpretation and a network implementation, which for gradient descent based optimization exploits a reparametrisation trick



# Summary of Deep Generative Models

## Resources:

- <https://huggingface.co/learn/computer-vision-course/en/unit5/generative-models/introduction/introduction>
- [https://huggingface.co/learn/computer-vision-course/en/unit5/generative-models/variational\\_autoencoders](https://huggingface.co/learn/computer-vision-course/en/unit5/generative-models/variational_autoencoders)
- <https://lilianweng.github.io/posts/2018-08-12-vae/>
- <https://www.jeremyjordan.me/autoencoders/>
- <https://www.jeremyjordan.me/variational-autoencoders/>
- <https://www.cnblogs.com/wangxiaocvpr/p/11605989.html>
- <https://medium.com/sciforce/generative-models-under-a-microscope-comparing-vaes-gans-and-flow-based-models-344f20085d83>
- <https://medium.com/sciforce/generative-models-under-a-microscope-comparing-vaes-gans-and-flow-based-models-344f20085d83>



# Parameter Efficient Fine-Tuning (PEFT)

